Assignment 1 – Static Code Analysis Using PMD

Module:	**Quality Assurance and Software Testing**	**Level:**	**Master 2 – Software Engineering**
Tool:	**PMD**	**Deadline:**	**Tuesday, November 3rd, 2026**

1. Assignment Overview

In this assignment, you will perform static code analysis on Java source code using PMD.

Three Java classes are available in the GitHub repository, inside the folder named `Assignment1`.

You must choose **ONE class** among the three available classes and perform a complete code analysis using PMD.

2. Work to Be Done

Part 1: Initial Code Analysis

1. Clone or update the GitHub repository to obtain the latest assignment files or download the folder directly.
2. Navigate to the `Assignment1` folder.
3. Choose one Java class to analyze.
4. Launch the terminal and execute the PMD commands studied during the previous laboratory session and documented in the second lab paper.
5. Generate the PMD analysis reports in the following two required formats:
   - Text
   - HTML

> **Note on Report Formats**
> Generating additional formats, such as XML or CSV, is optional and will be considered an additional effort.

Part 2: Analysis of PMD Results

Based on the initial reports, answer the following questions:

1. How many violations did PMD detect in your chosen class?
2. Identify and classify the detected violations according to the following PMD categories:

Best Practices	**Code Style**	**Design**	**Documentation**
Error Prone	**Multithreading**	**Performance**	**Security**

3. Which category contains the highest number of violations?

Part 3: Limitations of Static Code Analysis

PMD performs static code analysis. However, can PMD detect all errors and defects present in a program?

Discuss the limitations of PMD by considering the following aspects:

- **Visual analysis:** Can code analysis tools identify all problems related to the user interface and visual appearance of an application?
- **Execution analysis:** Can static analysis identify all defects that may appear only when the program is executed?

Part 4: Code Improvement

1. Correct the violations identified by PMD in your selected Java class.
2. Ensure that your modifications do not introduce new defects or change the intended behavior of the program.
3. Save the corrected version of your Java class.
4. Execute PMD analysis again using the same analysis configuration.
5. Generate the final PMD reports in the following two required formats:
    - Text
    - HTML

> **Report Clarification**
> - **Initial PMD report:** The report generated before correcting the detected violations.
> - **Final PMD report:** The report generated after correcting the detected violations.

Part 5: Results and Comparison

Present a comparison between the initial and final analysis results, including:

- Total number of violations before correction.
- Total number of violations after correction.
- Number of violations corrected.
- Remaining violations, if any.
- Changes observed in the different PMD categories.

Provide a brief conclusion about the improvements achieved and the effectiveness of PMD in improving code quality.

3. Deliverables

All submission files must be organized inside **ONE folder**.

Your folder must contain:

1. **Original Java class:** The selected class before correction.
2. **Corrected Java class:** The class after correcting the violations.
3. **Initial PMD report:** The HTML report generated before correcting the violations.
4. **Final PMD report:** The HTML report generated after correcting the violations.
5. **Assignment document:** A document presenting:
    - The names of the two students.
    - The selected Java class.
    - The answers to all assignment questions.
    - The classification of violations by PMD category.
    - The analysis of the detected violations.
    - The comparison between initial and final results.
    - The conclusion.

> **Note:** Only HTML reports are mandatory for submission. Additional report formats (TXT, CSV, XML, etc.) will be considered and appreciated.

4. Submission Rules

- Work must be completed in pairs (**two students only**).
- Individual submissions and groups of three students will **not** be accepted.
- Each pair must choose and analyze one Java class.
- The assignment document must clearly mention the full names of both students.
- All deliverables must be placed inside one folder.

> **Deadline:** Tuesday, November 3rd, 2026.